Агенты и LangGraph

01 Кто выбирает следующий шаг

Возьмём один вопрос **«Можно вернуть заказ №42?»**.

Подход	Кто задаёт маршрут
Chain	Код заранее задаёт шаги. Проверить заказ → найти правило → ответить.
Workflow	Код задаёт маршруты и условия. Если заказ не найден, спросить номер. Если найден, проверить правило.
Agent	Модель выбирает следующий инструмент по задаче и предыдущим результатам. Код ограничивает доступные действия.

ReAct - выбрать действие, получить результат, снова выбрать действие или ответить. **Plan-and-Execute** - сначала составить план из нескольких шагов, потом выполнять и при необходимости уточнять его.

Сравнить подходы на одной задаче.

02 Что находится в State

State - рабочие данные текущего диалога или задачи. **Node** - функция, которая читает эти данные и возвращает обновление. Например, `check_order` добавляет результат проверки.

```
# До узла
{"order_id": 42, "attempts": 0}
# Узел вернул только изменённые поля
{"attempts": 1, "age_days": 40}
# После объединения
{"order_id": 42, "attempts": 1,
 "age_days": 40}
```

Reducer задаёт способ объединения значений одного поля. По умолчанию новое значение заменяет старое. Для сообщений `add_messages` добавляет новые ID и заменяет сообщение при совпадении ID. Поля, которые узел не вернул, сохраняются.

Посмотреть State и reducer в коде.

03 Куда идёт граф

Edge соединяет шаги. **Conditional edge** выбирает маршрут по состоянию. Например, роутер читает последнее сообщение модели.

```
START → model
model → END    (лимит исчерпан)
model → END    (нет вызова инструмента)
model → tools  (есть вызов и остались попытки)
tools → model  (результат записан в State)
```

В ручном цикле Python это были `while`, проверка `tool_calls` и вызов функции. В **StateGraph** это узлы и рёбра. `compile()` собирает запускаемый граф, `invoke()` запускает его с входными данными.

Лимит попыток в State даёт контролируемый выход. `recursion_limit` служит запасным ограничителем. Если граф превышает допустимое число шагов, возникает `GraphRecursionError`.

Агенты и LangGraph

04 Где сохраняется прогресс

Термин	Что означает
Checkpoint	Снимок состояния и прогресса выполнения на границе шага графа.
Checkpointer	Механизм записи и чтения снимков. Память удобна для демо, постоянное хранилище переживает перезапуск процесса.
`thread_id`	Ключ диалога или задачи, по которому загружаются её снимки.

Если открыть тот же `thread_id`, граф может продолжить с сохранёнными данными. Новый ID начинает отдельную историю, если для него ещё нет снимка.

State и запрос к модели - два разных набора данных. Граф может хранить заказ, счётчики и историю. В модель попадут только сообщения и другие данные, которые код включил в запрос.

Посмотреть, как сохраняется State.

05 Как поставить граф на паузу

Например, помощник подготовил возврат заказа №42. Перед операцией просим человека проверить её.

```
Подготовить → interrupt(черновик)
                ↓ пауза и снимок
Человек → одобрить / исправить / отклонить
                ↓ тот же thread_id
Command(resume=решение) → выбранная ветка
```

Для паузы нужен checkpointer. При возобновлении **узел запускается с начала**. Дойдя до `interrupt()`, он получает значение из `resume` и продолжает по коду.

После правки проверяем новые параметры. После отказа заканчиваем без операции. После одобрения переходим к отдельному узлу исполнения.

06 Как не провести возврат дважды

Если до `interrupt()` стоит запись во внешнюю систему, повторный запуск узла может выполнить её снова. Checkpoint фиксирует ход графа, а внешний сервис живёт по своим правилам.

Идемпотентность означает, что повтор той же операции сохраняет тот же результат без дубля. Например, `refund:order-42:req-17` - один и тот же ключ при каждой попытке провести конкретный возврат. Сервис возвратов проверяет ключ атомарно и возвращает прежний результат при повторе.

Проверка руками. Запусти запрос, останови граф на подтверждении, затем проверь ветки одобрения и отказа. Смоделируй сбой сразу после записи и повтори тот же запрос. У возврата должна остаться одна запись.

Разобрать повтор запуска.